



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

CƠ SỞ HẠ TẦNG CÔNG NGHỆ THÔNG TIN

Biên soạn: ThS. Nguyễn Thị Anh Thư

Email: thunta@uit.edu.vn



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

Chương 4

Phần mềm hệ thống và phần mềm ứng dụng

Biên soạn: ThS. Nguyễn Thị Anh Thư

Email: thunta@uit.edu.vn

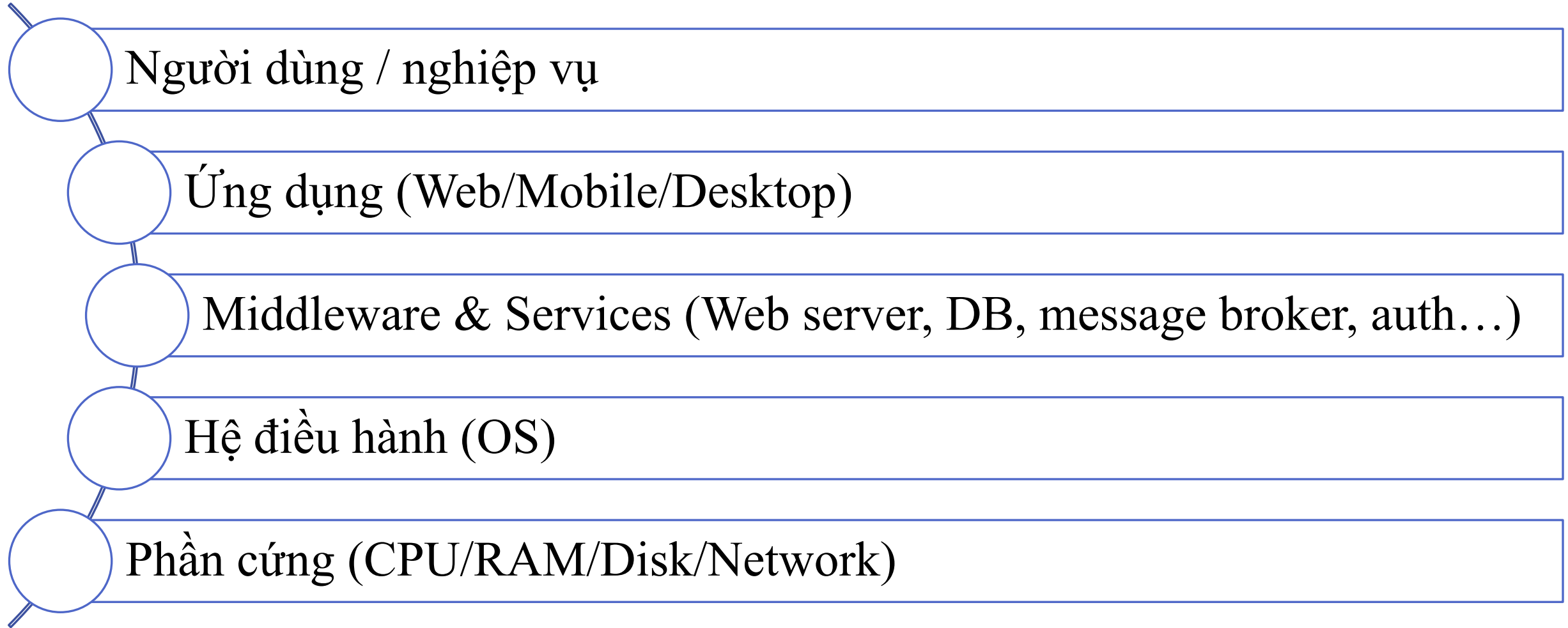


Nội dung

1. Bức tranh tổng thể: “Software Stack”
2. Hệ điều hành (Operating System)
3. Phần mềm hệ thống (System Software)
4. Phần mềm ứng dụng (Application Software)
5. Middleware & Services
6. Ảo hóa (Virtualization)
7. Containers
8. Quản lý vòng đời phần mềm
9. Tương thích và phụ thuộc
10. Bản quyền phần mềm & tuân thủ



1. Bức tranh tổng thể: “Software Stack”



2. Hệ điều hành (Operating System)

OS = lớp trung gian giữa phần cứng và phần mềm

Cung cấp “môi trường chạy” + quản trị tài nguyên

Hai nhóm đối tượng chính:

- Người dùng/ứng dụng
- Thiết bị phần cứng



3. Phần mềm hệ thống (System Software)

Bao gồm:

- OS (Windows/Linux/macOS), kernel, shell
- Device drivers
- Utilities: backup, antivirus, disk tools, monitoring agents
- Runtime/Platform: JVM, .NET runtime (tùy ngữ cảnh có thể coi nền tảng)

Mục tiêu: duy trì vận hành ổn định và cung cấp dịch vụ nền

4. Phần mềm ứng dụng (Application Software)

Phục vụ nhu cầu người dùng/đơn vị:

- Văn phòng: Office, email client
- Nghiệp vụ: ERP/CRM, LMS, HIS
- Web/Mobile app, SaaS (Google Workspace, Microsoft 365...)

Đặc trưng:

- Thường phụ thuộc OS + middleware/platform
- Có vòng đời cập nhật theo tính năng + bảo mật



5. Middleware & Services

Middleware = lớp “kết nối & điều phối” giữa ứng dụng và hệ thống

Cung cấp năng lực dùng chung:

- Kết nối (HTTP, RPC), session, caching
- Tích hợp (API Gateway, ESB)
- Messaging (queue/stream)
- AuthN/AuthZ (SSO, IAM)
- Logging/metrics/tracing (quan sát hệ thống)



5. Middleware & Services

Service/Daemon = chương trình chạy nền, cung cấp chức năng qua network/IPC

Đặc điểm:

- Có port/endpoint, có health check
- Có log, cấu hình, quyền truy cập
- Có SLA nội bộ (uptime, latency)

Tư duy hiện đại:

- **“Everything as a service” trong nội bộ: auth, file, monitoring...**



6. Ảo hóa (Virtualization)

Khái niệm: tạo máy ảo (VM) chạy trên phần cứng thật

Thành phần:

- Hypervisor (Type 1/Type 2)
- VM có OS riêng + kernel riêng

Lợi ích:

- Cách ly mạnh, tận dụng tài nguyên, dễ snapshot/backup

Hạn chế:

- Tốn tài nguyên hơn container, khởi động chậm hơn



7. Containers

Khái niệm: “đóng gói” ứng dụng + dependency thành image, chạy thành container

Chia sẻ kernel của host OS (khác VM)

Lợi ích:

- Nhẹ, khởi động nhanh, triển khai đồng nhất Dev/Test/Prod

Hạn chế:

- Cách ly thường “mỏng hơn VM” (tùy cấu hình), phụ thuộc kernel/OS family



8. Quản lý vòng đời phần mềm

Mục tiêu: ổn định + an toàn + tuân thủ

Thành phần vòng đời:

- Lập kế hoạch phiên bản
- Triển khai (deploy)
- Vận hành/giám sát
- Cập nhật/vá lỗi
- Kết thúc vòng đời & thay thế



9. Tương thích và phụ thuộc

Nguồn lỗi phổ biến:

- Thay đổi API/phiên bản thư viện
- Hệ điều hành không còn hỗ trợ (EOL)
- Driver/firmware không tương thích
- Cấu hình môi trường khác nhau Dev/Test/Prod

Cách kiểm soát:

- Versioning rõ ràng, release notes
- Test hồi quy (regression)
- Chuẩn hóa môi trường (image/container), “pin version”



10. Bản quyền phần mềm & tuân thủ

Vì sao quan trọng:

- **Rủi ro pháp lý, kiểm toán, chi phí phát sinh**

Nhóm giấy phép thường gặp:

- Proprietary (Windows, Office...)
- Open-source (MIT/Apache/BSD; copyleft như GPL)
- Subscription/SaaS (theo user/thiết bị/tài nguyên)

Thực hành quản trị:

- Inventory phần mềm (asset list)
- Quản lý key/license/subscription
- Theo dõi điều khoản OSS trong dự án

Q & A
